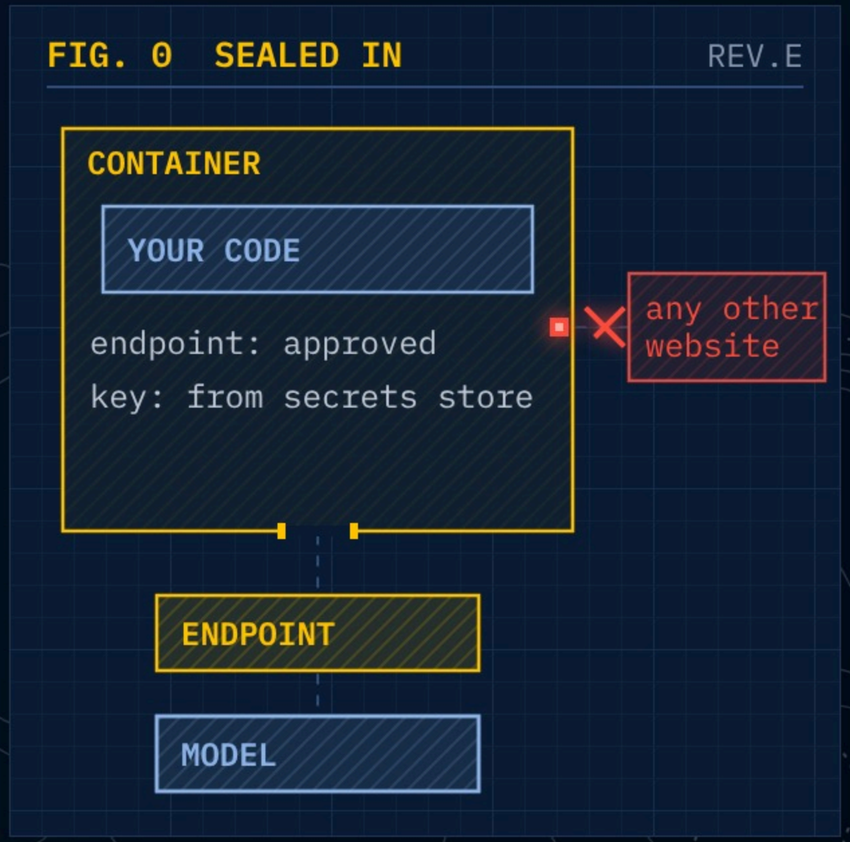
Kinvecta

TRAIN-THE-TRAINER SERIES · SESSION 5 OF 8

APIs, endpoints and containers

When your own software calls the model, your team owns the controls.



6–minute video | Stage 2 | Guide 2.1 · 3.2

Kinvecta

TODAY

By the end, your learners can…

1. Explain what changes when your code calls the model
2. Choose an endpoint and keep it in config
3. Describe what a container allows and blocks
4. Name the four run controls

CHAPTERS · 6 MIN

drawn to scale



Kinvecta

WHY THIS MATTERS

Three problems that start the day code calls the model

01

“The key was in the script”

A pasted API key ends up in code, in the repository and in the logs.

02

“We can’t switch vendors”

The endpoint and model name are hard-coded in fifty places.

03

“Nobody knows what ran”

No record of which prompt, which model version, or who reviewed it.

The question for this session:

Who owns the controls when code calls the model?

Kinvecta

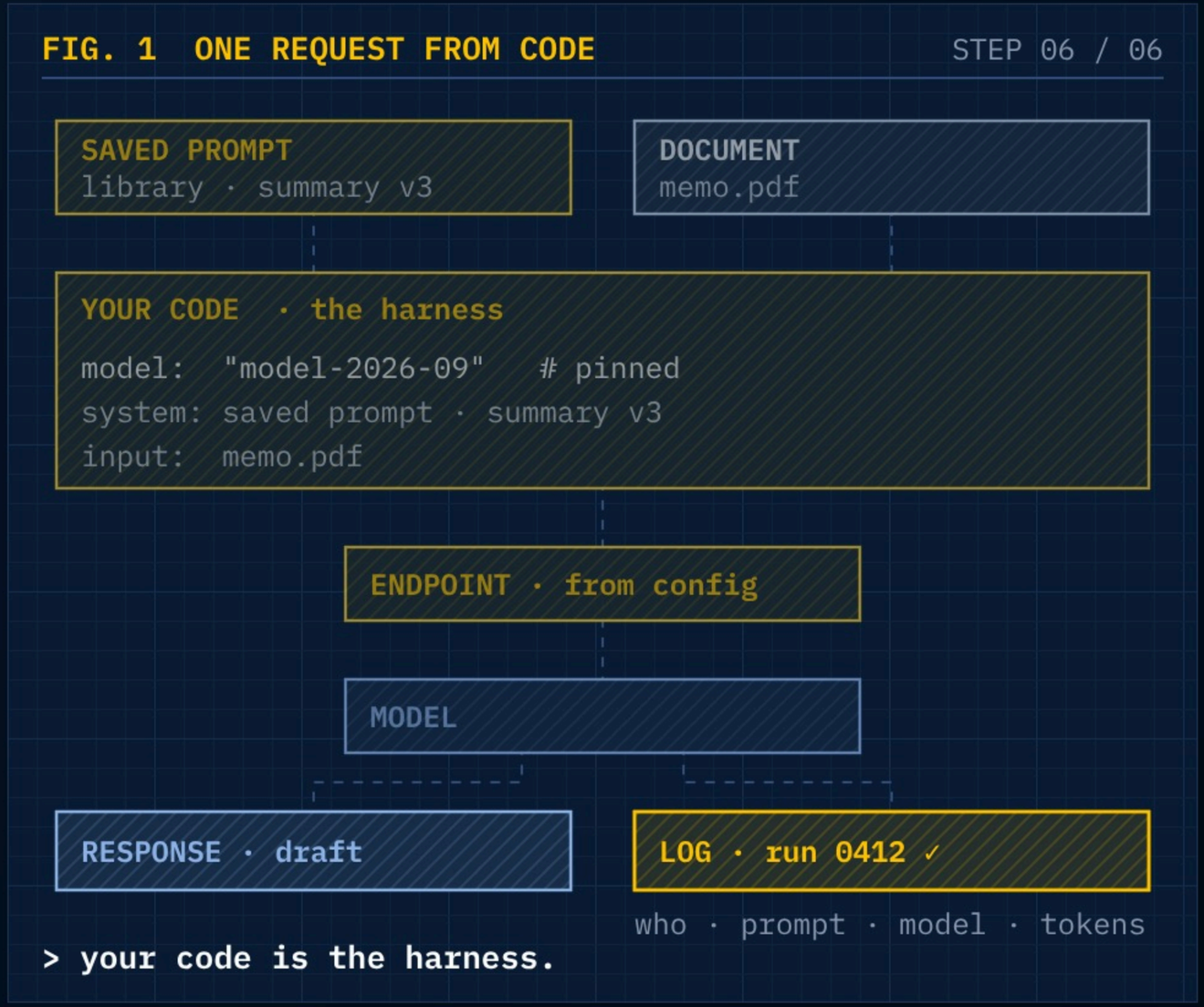
THE API

Through the API, your software becomes the harness

Same prompt, sent from code
Load the saved prompt from the library; don't rewrite it in code

You own the controls
Instructions, data handling, logging and security are now yours

Different terms
Retention, authorization and pricing differ from the chat app



Kinvecta

WHAT YOUR TEAM TAKES ON

The chat app did six jobs for you. Now your code does.

Watch each job move across. The numbers **01–06** come back on every figure that follows.



Kinvecta

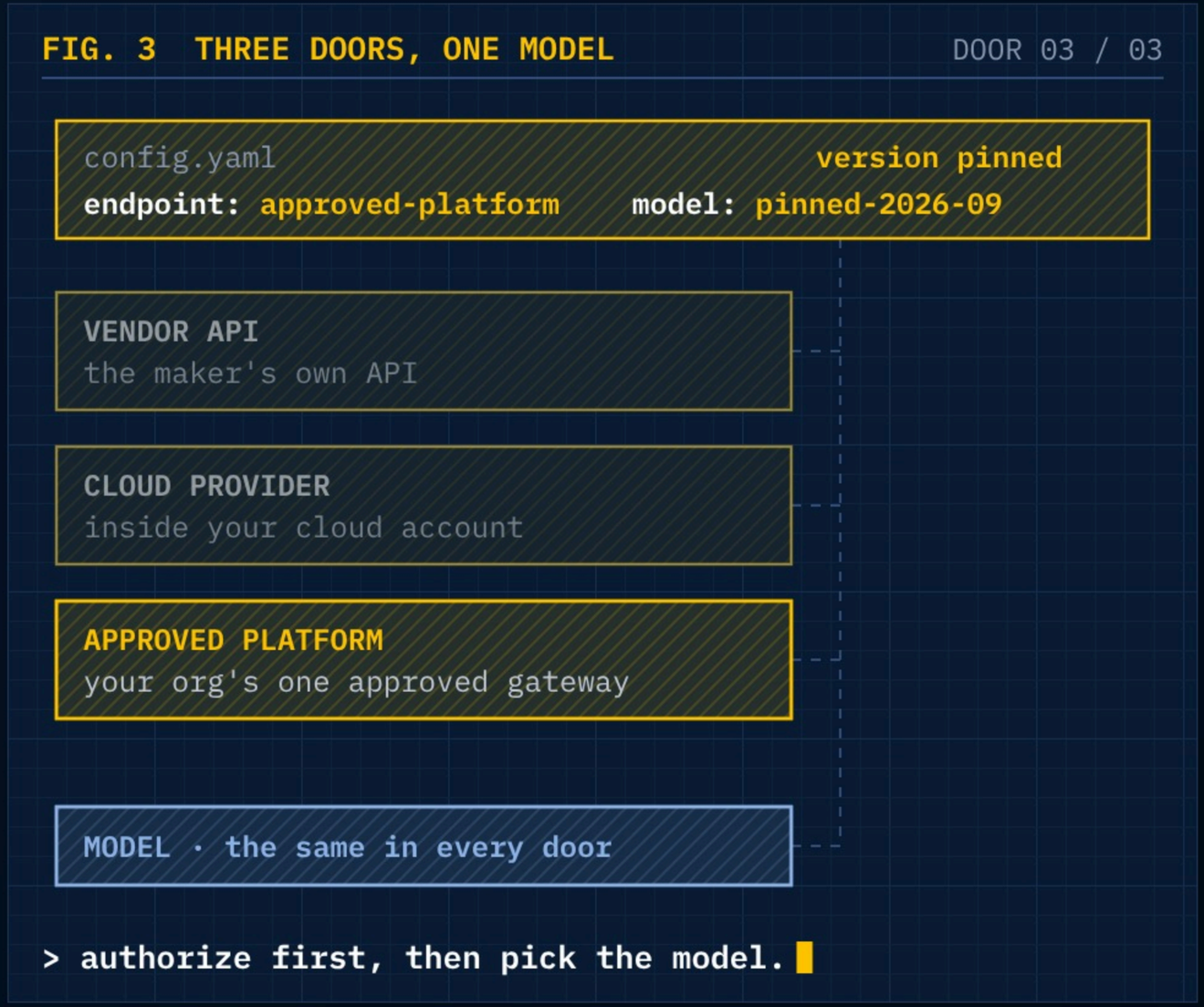
WHERE THE API COMES FROM

Three front doors to the same model

Same model, different contract
Authorization, data handling and terms change by door

Authorization first
Pick the door by data level and approval, then the model

Keep it in config
Switching doors is a one-line change



Kinvecta

KEEP THE ENDPOINT IN CONFIG

Four habits that make a switch a one-line change

01

Endpoint in config

Switching is a one-line change, not a code edit

02

Pin the version

Never “latest”; re-run the tests before you switch

03

One request shape

Write the request the same way for every door

04

Allow-list it

Make it the container’s only permitted address

Kinvecta

CONTAINERIZATION

Every harness and workflow runs sealed in a container

Reviewed libraries only

The exact libraries that were reviewed are the ones that run

One way out · 04

Its only permitted address is the approved model endpoint

One image to authorize

The same scanned image is authorized and redeployed



Kinvecta

RUN CONTROLS

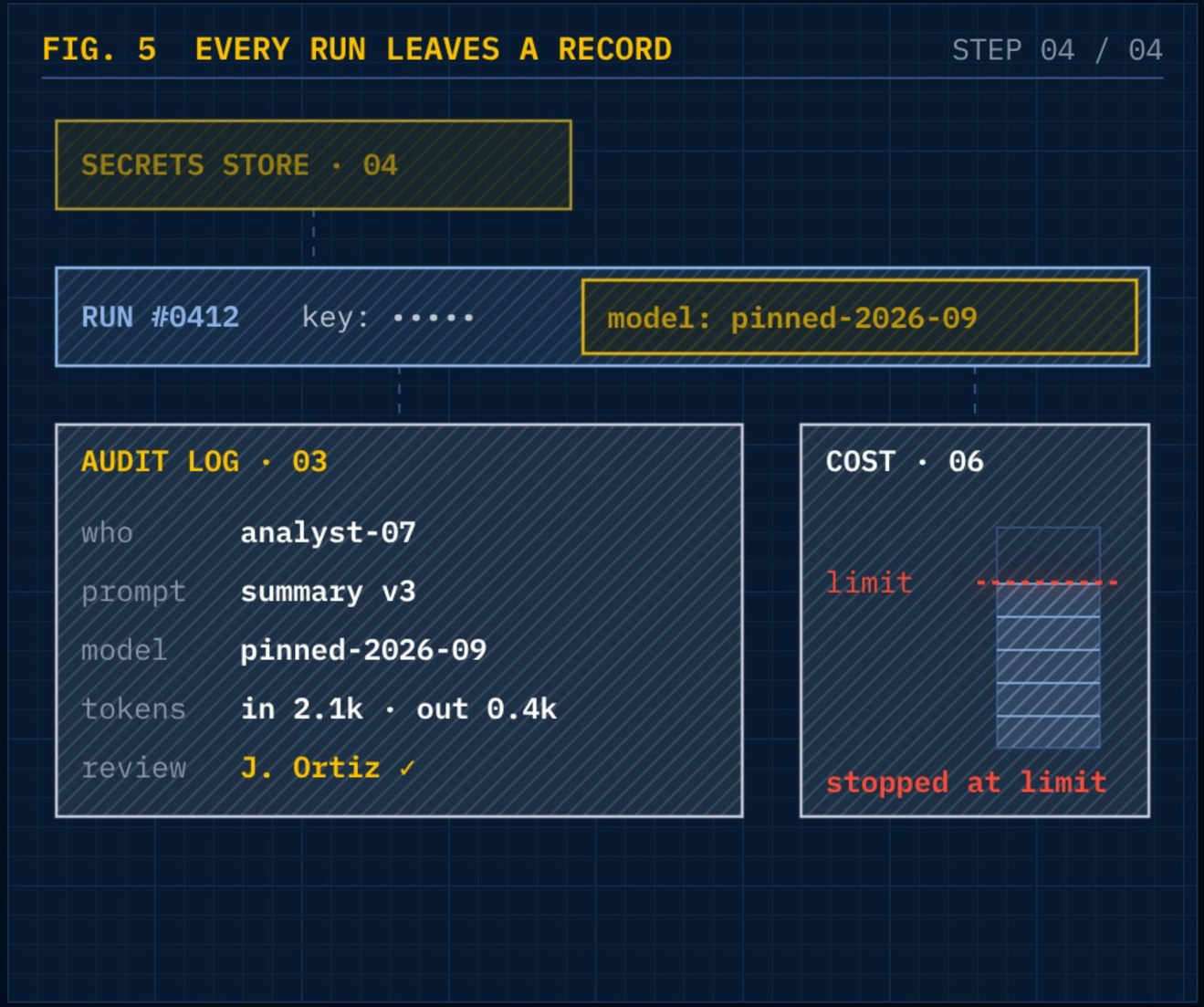
Every run leaves a record

Secrets at run time · 04
Never in code, prompts or logs

Pin the model version
Switch on purpose, after tests pass

Log every run · 03
Who, which versions, inputs, outputs

Rate and cost limits · 06
A loop can’t run up the bill



Kinvecta

WHAT IS AVAILABLE TODAY

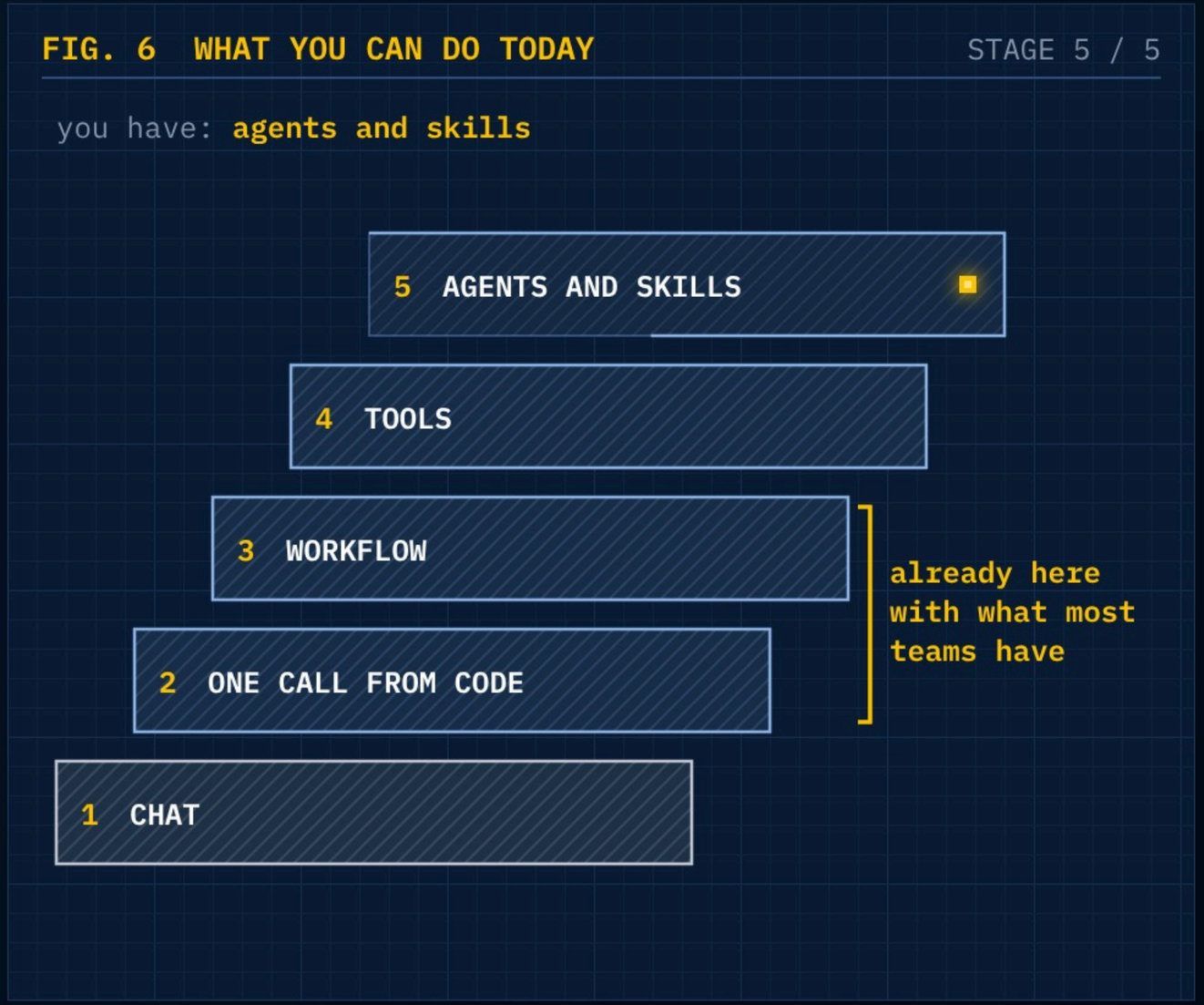
Chat plus an API already gets a team to Stage 3

Chat only
Stage 1: prompts and templates

Chat + API
Stages 2–3: workflows from code

+ tool calling
Stage 4: approved lookups and actions

Agents and skills
Stage 5: multi-step, approval gates



Kinvecta

THE RULE OF THUMB

Who owns the controls? **You do.**

Call it from code.
Keep the endpoint in **config**.
Seal it in a **container**.

Your code becomes the harness, so your team owns the controls.

Kinvecta

EXERCISE

Read it, then box it

Pause & try

A Read it · find five problems

PSEUDOCODE

```
1 API_KEY = "sk-4f9a…"
2 url = "https://api.vendor.example/v1"
3 model = "latest"
4 prompt = "Summarize this: " + doc
5 reply = send(url, API_KEY, model, prompt)
6 print(reply)
```

doc = the file being summarized · hints: key, endpoint, model, prompt, reply

B Box it

A weekly contract-summary workflow.

CONTAINER

INSIDE _____

ONE ENDPOINT _____

KEY FROM _____

EACH RUN LOGS _____

LIMIT _____

Kinvecta

CHECK FOR UNDERSTANDING

Can every learner answer these?

1

When your code calls the model instead of the chat app, what does your team now have to handle?

2

Name the three kinds of endpoint.

3

Which do you choose first, the door or the model, and why?

4

Why keep the endpoint in config?

5

What can a container reach?

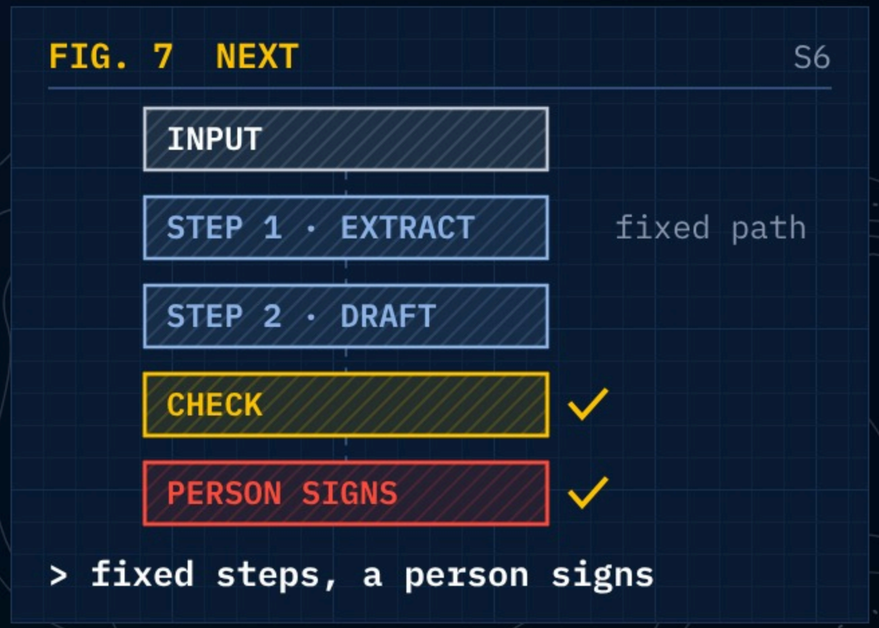
6

Name the four run controls.

Kinvecta

NEXT · SESSION 6 · BUILDING WORKFLOWS

How do you turn a repeat task into a fixed pipeline?



What Session 6 covers

- Fixed steps
- One check
- Human review
- Saved prompts, loaded by code

Kinvecta

APPENDIX · FOR FACILITATORS

Build the Stage 2 sandbox once, reset it per cohort

IMAGE

Base container image

Pre-scanned; network limited to the training endpoint

KEYS

Training API keys

Per-student, low limits, from the secrets store, revoked after class

DOCS

Sample documents

Made-up memos and reports; nothing real

LIBRARY

Starter prompt library

A Git repository with approved prompts and test cases

RULE Class time goes to learning, not setup: every Stage 2–5 exercise runs in the same prepared, non-production sandbox.